



Програмиране за мобилни Интернет устройства

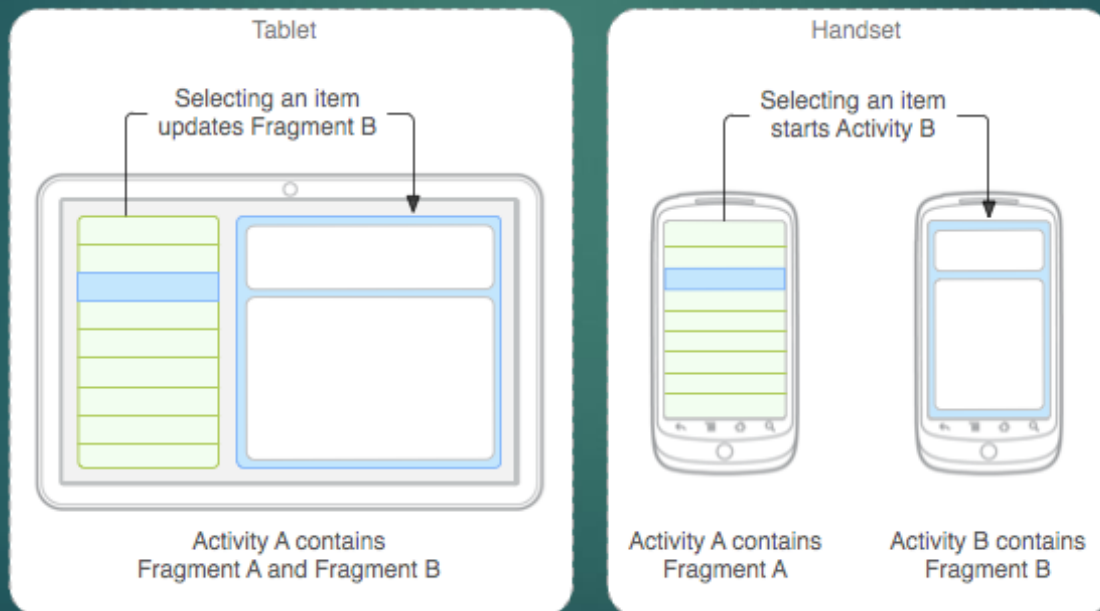
ФРАГМЕНТ. ЖИЗНЕН ЦИКЪЛ НА ФРАГМЕНТ.
УПРАВЛЕНИЕ И ВЗАИМОДЕЙСТВИЯ НА ФРАГМЕНТИ

Проблеми на Активити

- ▶ Проблем с размера на екрана
- ▶ Проблем при промяна на конфигурация

Фрагменти

- ▶ Android 3.0 (API level 11) – в отговор на появата на мобилни устройства с големи екрани
- ▶ Възможност за промяна на изглед и поведение по време на работа на приложението



Фрагмент. Същност

- ▶ Представява част от поведението и графичния интерфейс на едно активити
 - ▶ Задължително част от активити
- ▶ В комбинация – реализация на по сложни активити компоненти
- ▶ Добавя се към View групата на активиралото го активити
 - ▶ Дефинира и собствен изглед или група от изгледи
 - ▶ Добавя се посредством декларация на `<fragment>` в схемата на изгледа или от Java кода на приложението чрез добавяне към текущ дефиниран изглед

Фрагмент. Същност

- ▶ Концепция на модула – капсулиране и сепарация
 - ▶ Самостоятелен жизнен цикъл **в рамките на активити**
 - ▶ Собствено прихващане и обработване на събития
 - ▶ Възможност за активиране (прикачване) и деактивиране (откачане) докато съдържащото го активити работи
 - ▶ Концепция за преизползване на код
 - ▶ Един фрагмент може да се активира от различни активити

Създаване на фрагменти

- ▶

```
public static class ExampleFragment extends Fragment {  
    @Override  
    public View onCreateView(LayoutInflater inflater, ViewGroup  
container, Bundle savedInstanceState) {  
        return inflater.inflate(R.layout.example_fragment, container, false);  
    }  
}
```
- ▶ Методът *inflate*

ДОПЪЛНИТЕЛНИ КЛАСОВЕ

- ▶ DialogFragment
 - ▶ Аналогия с dialog helper, но добавя в стека на фрагментите
- ▶ ListFragment
 - ▶ При работа с адаптери
- ▶ PreferenceFragmentCompat
 - ▶ За визуализация на обекти от тип Preference

Добавяне на фрагмент.

Статичен подход

```
► <?xml version="1.0" encoding="utf-8"?>
  <LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <fragment android:name="com.example.news.ArticleListFragment"
      android:id="@+id/list"
      android:layout_weight="1"
      android:layout_width="0dp"
      android:layout_height="match_parent" />
    <fragment android:name="com.example.news.ArticleReaderFragment"
      android:id="@+id/viewer"
      android:layout_weight="2"
      android:layout_width="0dp"
      android:layout_height="match_parent" />
  </LinearLayout>
```


Добавяне на фрагмент. Статичен подход

```
<androidx.fragment.app.FragmentContainerView  
    xmlns:android="http://schemas.android.com/apk/res/android"  
    android:id="@+id/fragment_container_view"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    android:name="com.example.ExampleFragment" />
```

Добавяне на фрагмент.

Динамичен подход

- ▶ По време на изпълнение
 - ▶ Необходимост от дефиниране на placeholder – ViewGroup
 - ▶ `<androidx.fragment.app.FragmentContainerView`
xmlns:android="http://schemas.android.com/apk/res/android"
android:id="@+id/fragment_container_view"
android:layout_width="match_parent"
android:layout_height="match_parent" />
- ▶ Принцип на транзакциите – добавяне, премахване, подмяна
 - ▶ API – `FragmentManager`

Добавяне на фрагмент.

Динамичен подход

```
▶ public class ExampleActivity extends AppCompatActivity {  
    public ExampleActivity() {  
        super(R.layout.example_activity);  
    }  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        if (savedInstanceState == null) {  
            getSupportFragmentManager().beginTransaction()  
                .setReorderingAllowed(true)  
                .add(R.id.fragment_container_view, ExampleFragment.class,  
null)  
                .commit();  
        }  
    }  
}
```

При необходимост от входна информация

```
▶ public class ExampleActivity extends AppCompatActivity {  
    public ExampleActivity() {  
        super(R.layout.example_activity);  
    }  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        if (savedInstanceState == null) {  
            Bundle bundle = new Bundle();  
            bundle.putInt("some_int", 0);  
  
            getSupportFragmentManager().beginTransaction()  
                .setReorderingAllowed(true)  
                .add(R.id.fragment_container_view, ExampleFragment.class,  
bundle)  
                .commit();  
        }  
    }  
}
```

При необходимост от входна информация

```
▶ class ExampleFragment extends Fragment {  
    public ExampleFragment() {  
        super(R.layout.example_fragment);  
    }  
  
    @Override  
    public void onViewCreated(@NonNull View view, Bundle  
savedInstanceState) {  
        int someInt = requireArguments().getInt("some_int");  
        ...  
    }  
}
```

Транзакции на фрагментите. FragmentManager

- ▶ FragmentTransaction API
- ▶ Поредица от промени които трябва да са атомарни
 - ▶ add(), remove(), replace()
- ▶ addToBackStack() !!!
- ▶ commit()
 - ▶ Асинхронен
 - ▶ commitNow() – несъвместим с addToBackStack()
 - ▶ executePendingTransactions()
- ▶ Комплексни операции
 - ▶ Преди commit()
- ▶ Последователност на фрагментите

Замяна на фрагмент

```
FragmentManager fragmentManager = getSupportFragmentManager();  
fragmentManager.beginTransaction()  
    .replace(R.id.fragment_container, ExampleFragment.class, null)  
    .setReorderingAllowed(true)  
    .addToBackStack(null)  
    .commit();
```

...

```
ExampleFragment fragment =  
    (ExampleFragment)  
fragmentManager.findFragmentById(R.id.fragment_container);
```

Управление на фрагменти. FragmentManager

- ▶ `getSupportFragmentManager()`
- ▶ `findFragmentById()` – фрагмент с UI
- ▶ `findFragmentByTag()` – фрагмент без UI (headless fragment)
- ▶ `popBackStack()` – симулиране на Back

Взаимодействие с изгледите

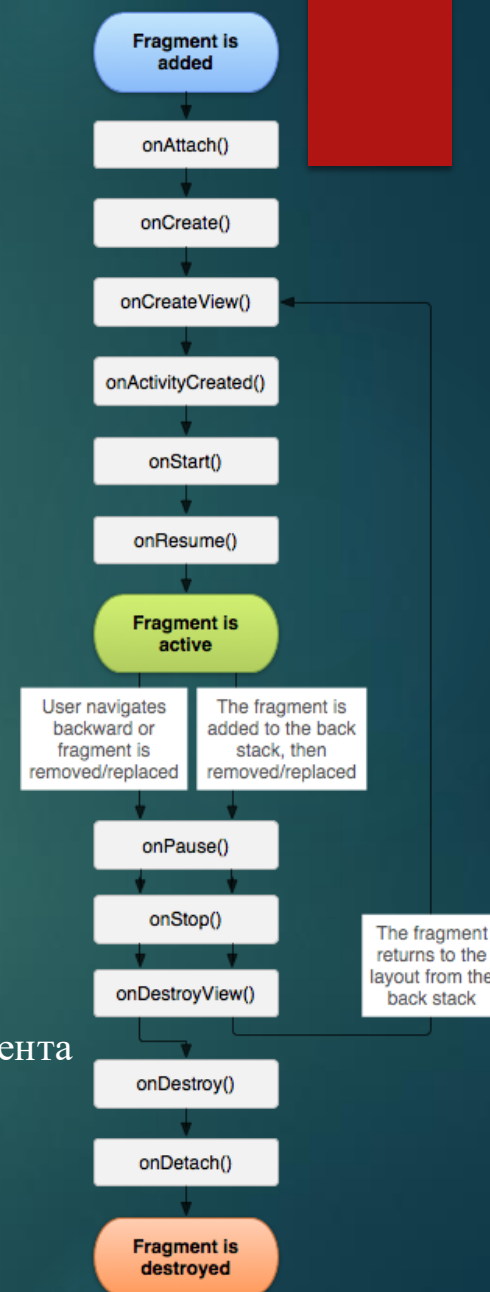
- ▶ detach()
 - ▶ STOPPED
- ▶ attach()

Жизнен цикъл на фрагмент

- ▶ INITIALIZED
- ▶ CREATED
- ▶ STARTED
- ▶ RESUMED
- ▶ DESTROYED

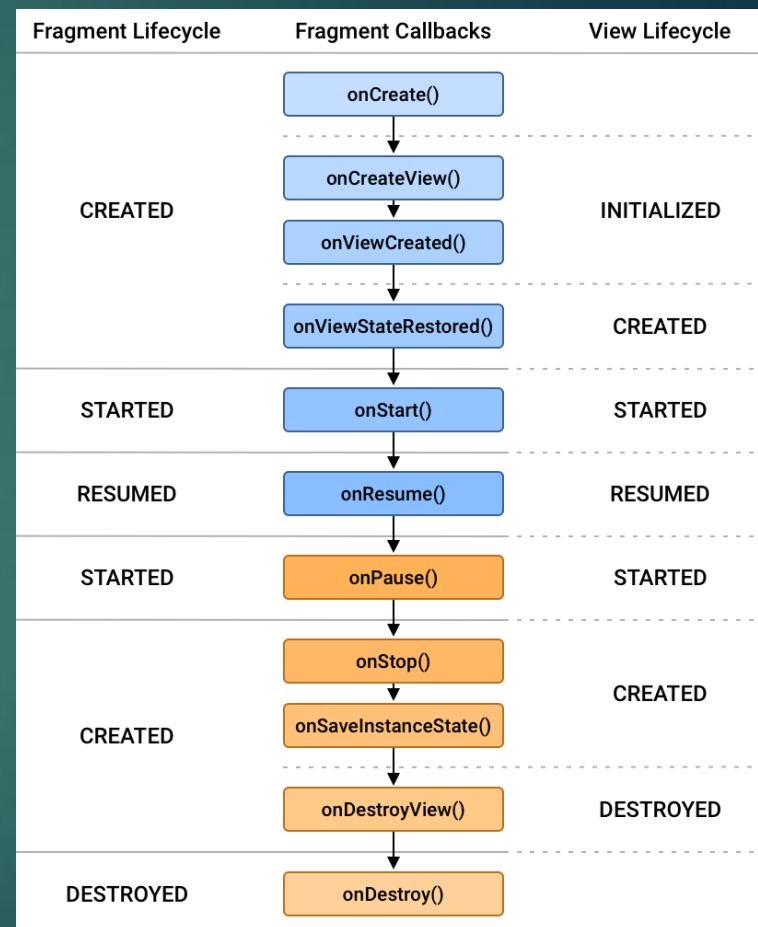
- ▶ onCreate(), onStart(), onResume(), onPause(), onStop(), onDestroy()

- ▶ onCreate() – аналогия с метода на активити
- ▶ onCreateView() – подготвяне и визуализация на фрагмента
- ▶ onPause() – извиква се при излизане от фрагмента
 - ▶ Не е задължително свързано с унищожаването на фрагмента
 - ▶ Запазване на данни

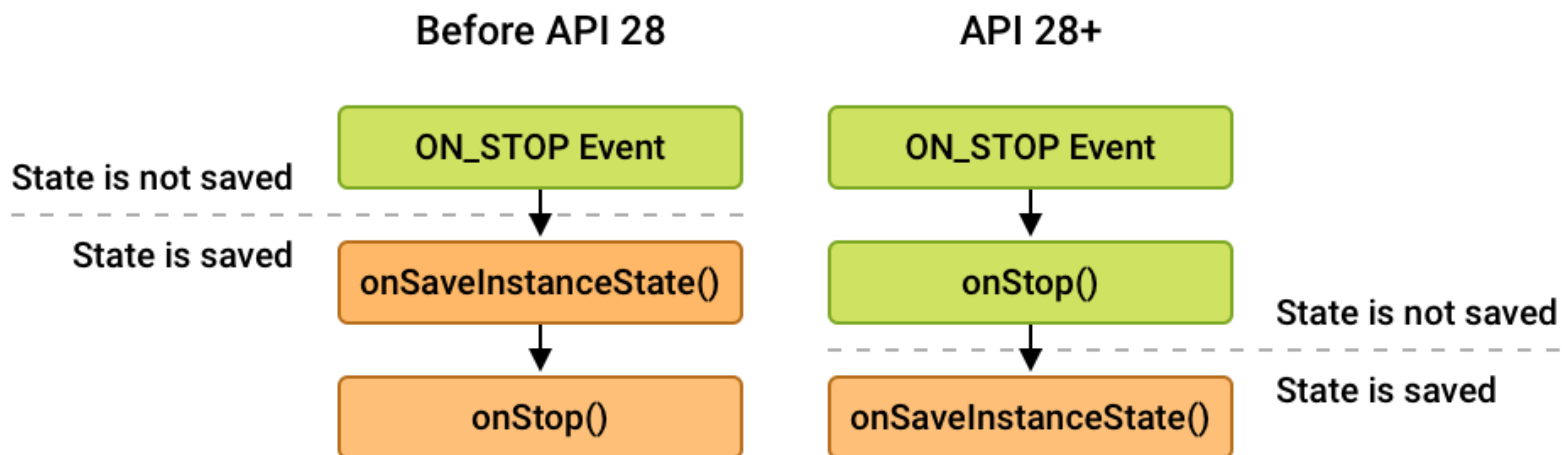


Жизнен цикъл на фрагмент

- ▶ Директно отражение на методите от Активити
- ▶ `onAttach()`
- ▶ `onCreateView()` – за създаване на йерархията от изгледи асоциирани с фрагмента
- ▶ `onActivityCreated()` – при повикване на `onCreate()` на съдържащото Активити
- ▶ `onDestroyView()` – при премахване на йерархията от изгледи
- ▶ `onDetach()` – при премахване на асоциацията между Активити и Фрагмента



Жизнен цикъл на фрагмент



КОМУНИКАЦИЯ С АКТИВИТИ

- ▶ Фрагмент -> Активити

- ▶ `View listView = getActivity().findViewById(R.id.list);`

- ▶ Активити -> Фрагмент

- ▶ `ExampleFragment fragment = (ExampleFragment) getSupportFragmentManager().findFragmentById(R.id.example_fragment);`

Споделяне на информация между Активити и Фрагмент

- ▶ Необходимост от предаване на данни между програмните елементи
 - ▶ ViewModel (LiveData)
 - ▶ Callback интерфейс
 - ▶ имплементиран от активити

Споделяне на информация между Активити и Фрагмент. ViewModel подход

```
▶ public class MainActivity extends AppCompatActivity {  
    private ItemViewModel viewModel;  
  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        viewModel = new ViewModelProvider(this).get(ItemViewModel.class);  
        viewModel.getSelectedItem().observe(this, item -> {  
            // взаимодействие с item  
        });  
    }  
}
```

Споделяне на информация между Активити и Фрагмент. ViewModel ПОДХОД

```
▶ public class ListFragment extends Fragment {  
    private ItemViewModel viewModel;  
  
    @Override  
    public void onViewCreated(@NonNull View view, Bundle  
savedInstanceState) {  
        super.onViewCreated(view, savedInstanceState);  
        viewModel = new  
ViewModelProvider(requireActivity()).get(ItemViewModel.class);  
  
        ...  
  
        items.setOnClickListener(item -> {  
            viewModel.select(item);  
        });  
    }  
}
```